

Android Agentic Developer Loop — Project Report

Android Agentic Developer Loop — Project Report

Package: `agentic-android-loop` (published on npm, `v1.0.2`) **Date:** 2026-06-25 **Prepared by:** Mahesh

Section 1 — Manager Summary

What this is

An AI-powered automation tool that takes an Android feature from a **Jira ticket** to a **submitted Pull Request** with minimal developer intervention — using Claude Code as the execution engine, Atlassian RoVo (Jira) and Figma as the only external integrations, and no custom backend or service of our own.

A developer types one sentence into Claude Code:

```
Run the agentic loop for ticket APP-101
```

and the tool reads the ticket, reads the linked Figma design, scans the existing codebase, generates the UI, writes the business logic and tests, runs all quality checks, and opens a PR — with the developer only stepping in at a handful of yes/no checkpoints.

Why this matters

- Reduces repetitive screen-building work (boilerplate ViewModel/Repository/tests/localisation) to an automated, auditable pipeline
- Enforces existing engineering standards automatically (no hardcoded strings, no hardcoded colors, MVVM/Hilt/StateFlow conventions, pixel-accurate design matching) rather than relying on code review to catch them after the fact
- Every run is traceable: Jira is updated at each stage, every flag/inference the agent made is surfaced in the PR description for human review — nothing is silently auto-approved
- Built entirely on tools we already pay for/use (Claude Code, Jira, Figma, GitHub Actions) — no new infrastructure, no new vendor, no custom API clients to maintain

Current status

Area	Status
Design (5-phase workflow spec)	✅ Complete
Helper tooling (pixel diff, string checker, localisation checker, PR builder)	✅ Built

Area	Status
CI workflow (test suite, APK size budget, localisation gate)	✅ Built
One-command installer, published to npm	✅ Live — <code>npx agentic-android-loop</code>
End-to-end run on a real ticket	🕒 Not yet executed
Rovo (Jira) MCP authentication	🕒 Needs to be connected per developer machine

In short: the system is fully built and installable today. It has not yet been run against a real ticket end-to-end — that is the natural next milestone before rolling out to the wider team.

Distribution model

The tool is installed **per Android repository**, not globally. Any developer with access to an existing Android project runs:

```
npx agentic-android-loop
```

from their project root, and it self-configures by reading the project's own Gradle files (auto-detects package name and source paths), merges safely into any `CLAUDE.md` / `.agentrc` the team already has (never overwrites existing config), and is fully idempotent (safe to re-run).

Once a team installs it and commits the generated files, every other teammate gets it automatically via `git pull` — they only need to run `npm install` once and confirm their own Jira/Figma authentication.

Open items before team rollout

1. Run a first real ticket through the full loop to validate the design end-to-end and surface any gaps in the instructions
2. Each developer needs Rovo (Jira) MCP authenticated individually — this is per-person, not shareable
3. `.agentrc` needs the real Jira project key and Figma file key filled in per-repo
4. Minor known inconsistency: the pixel-match pass/fail script has an internal hardcoded threshold separate from the configurable one — flagged for fix, not yet applied
5. The CI workflow's APK size baseline tracking needs the baseline file to be git-committed (not gitignored) to work correctly across runs — needs a decision on the team's process for updating it after merges

Section 2 — Developer / User Guide

1. What gets installed, and where

The tool installs into **whichever Android repo you run it from** — not globally. Running it inside a project adds:

```
your-android-project/
├── .agent/
│   ├── pixeldiff.js           # pixel-match comparison (pixelmatch + pngjs)
│   ├── string-checker.js      # scans .kt/.xml for hardcoded user-facing strings
│   ├── localisation-check.js  # verifies every string key exists in every language
│   ├── pr-builder.js          # assembles PR title/body/branch from the run payload
│   ├── screenshots/           # per-run UI renders + Figma exports (gitignored)
│   └── run_<TICKET_ID>.json    # shared state file each phase reads/writes (gitignored)
├── .agentrc                   # project config – paths, thresholds, Jira/Figma settings
├── .github/workflows/
│   └── agent-evaluate.yml      # CI: full test suite → APK size budget → localisation checks
├── CLAUDE.md                  # the operating instructions Claude Code follows (appends to your existing CLAUDE.md if you already had one)
├── package.json / package-lock.json # Node deps for the .agent/ scripts only – not part of the Android app build
└── .gitignore                 # node_modules/ added if not already present
```

Nothing here touches your actual Android source unless the loop is actually triggered and runs Design/Implement against a real screen — installation alone is config + tooling only.

2. Installing it

Open the Terminal tab in Android Studio (or any terminal at your project root) and run:

```
npx agentic-android-loop
```

This single command: - Detects your `app` module and reads `applicationId` / `namespace` from your Gradle files to auto-fill `packageName`, `sourceRoot`, and `sharedComponentsPath` in `.agentrc` - Creates `package.json` if you don't have one (with `pixelmatch`, `pngjs`, `glob` as deps) and runs `npm install` for you automatically - Adds the CI workflow if you don't already have one with that name - Appends the agentic-loop instructions to your existing `CLAUDE.md` (or creates one), inside a clearly marked block — re-

running the installer later updates just that block, never duplicates it - Warns you if you're not actually standing in an Android Studio project root (missing `build.gradle`)

Safe to run more than once — it never overwrites values you've already customized.

3. One-time config — `.agentrc`

After installing, open `.agentrc` and fill in what couldn't be auto-detected:

Key	What to set
<code>jira.baseUrl</code>	Your team's Atlassian URL
<code>jira.projectKey</code>	Your Jira project key (e.g. <code>APP</code>)
<code>figma.fileKey</code>	Your team's main Figma file key (or leave <code>null</code> to be prompted per ticket)
<code>languages</code>	Every locale you localise into, e.g. <code>["en", "hi", "mr"]</code>

Everything else (pixel match threshold, max retry attempts, APK size budget, folder paths) has sensible defaults already set.

4. Connecting Jira and Figma

This tool never calls the Jira or Figma REST APIs directly — it only goes through MCP. Check both are connected:

```
claude mcp list
```

You should see **Atlassian Roovo** and **Figma** both show “Connected.” If Roovo shows “Needs authentication,” authenticate it — this is per developer machine/account, not something that's shared via git.

5. Running the loop

In Claude Code, inside your project:

```
Run the agentic loop for ticket APP-101
```

You can also paste the full Jira URL instead of the bare ticket key — both work.

The 5 phases it runs through

#	Phase	What happens
1	Gate	Fetches the ticket, runs 4 checks (Figma link, AC completeness, blocked dependencies, scope size), detects the target screen, asks you to confirm, then moves Jira to “In AI Development”
2	Analyse	Scans your codebase + Figma + API contracts + existing strings in parallel, scores component reuse (Green/Yellow/Red), produces a task plan
3	Design	Generates the Compose/XML UI from Figma tokens, renders screenshots, pixel-diffs against the Figma export, loops fixes until the match threshold is hit — for every UI state (loading, error, empty, etc.)
4	Implement	Writes <code>UiState</code> , <code>ViewModel</code> , <code>Repository</code> , localized strings, and unit/UI tests — never touches files Design created
5	Evaluate	Runs build/lint/tests/string-check/pixel-recheck locally, loops back to Implement on failure (up to 3 times), then pushes to a <code>feature/**</code> branch, waits on CI, and opens the PR with Jira updated to “In Review”

What you’ll be asked along the way

- **Gate:** confirm the detected screen name; confirm soft-block overrides if AC is thin or multiple screens are detected; confirm before any Jira state change

- **Analyse:** if confidence is medium, confirm the plan; if low, review each task individually
- **Design:** if pixel match still fails after the max attempts, choose `[fix asset / accept / abort]`

Everything else runs unattended.

6. What you get at the end

- A PR with: feature summary, pixel match scores per UI state, screenshots at 3 breakpoints, every flag the agent raised during normalisation, and a localisation notice if translation placeholders were added
- Jira ticket moved to “In Review” with the PR linked as a comment
- If anything failed 3 times in a row, instead: Jira moved to “Needs Refinement” with a comment explaining what blocked it — nothing fails silently

7. Team onboarding (after one person has installed + pushed)

Everyone else just needs:

```
git pull
npm install
claude mcp list    # connect Rovo/Figma individually if needed
```

No one else needs to run the installer again — `.agentrc`, `CLAUDE.md`, and the helper scripts are already in the repo once committed.